

RBE 3001 - Team 3 Final Report

Evan Kaba

*Department of Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA, USA
ejkaba@wpi.edu*

Dhruv Madan

*Department of Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA, USA
dmadan@wpi.edu*

Brian Grande

*Department of Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA, USA
bmgrande@wpi.edu*

Luhai Tang

*Department of Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA, USA
ltang@wpi.edu*

Warren Yun

*Department of Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA, USA
wyun@wpi.edu*

I. ABSTRACT

This report explores the development of a robotic system designed to identify, pick up, and sort objects using a combination of computer vision and motion control. The project involved designing a framework and an operation method to effectively coordinate system tasks. At the system's core, computer vision is used to locate and identify objects, and a combination of forward and inverse kinematics is used to execute precise movements. This report will cover the process of iterations and development, explaining step by step the process, and reasoning behind the techniques and methods chosen. In addition, it presents an in-depth performance evaluation, noting object recognition accuracy, positional precision, and overall efficiency of the pick-and-place system. Through visual data representations such as graphs and diagrams, the report provides a comprehensive breakdown of the implementation and effectiveness of the system, demonstrating the seamless integration of robotic control and computer vision.

II. INTRODUCTION

Many automation tasks in the real world revolve around the concept of manipulation: being able to pick up objects of relevance and move them to desired locations without external input from a human. In the past decade, the world has seen more and more of these manipulators being sold on the market, like Boston Dynamics' Stretch or Tesla's Optimus, which are extended applications of this concept showcasing full-body manipulation and perception.

This project draws inspiration from such industrial robots, where automation is crucial for effective processes, while maintaining speed, accuracy, and reliability. During the past seven weeks through this project, our team was presented with a comprehensive task that took us through an invaluable learning experience filled with system design, computer vision, forward and inverse kinematic calculations, and programming abilities. Building upon skills and knowledge gained through lectures, we gained hands-on experience each week through foundational laboratory exercises, designed to combine and

build into our final project and the demonstration of our pick-and-place four-degree-of-freedom robotic arm.

A. Lab 1

In lab one, we were introduced to the OpenManipulator-X robot (OMX), this serial manipulator is a four degrees-of-freedom (DoF) arm. Which is built upon four DYNAMIXEL motors that rotate to control four rotational joints. As well as a small gripper servo that only provides opening and closing for grabbing our objects. Each of these motors are capable of position, velocity, and current controlling. Within this lab, we established the base actuation and sensor capabilities of our OpenManipulator X. This is carried out by the Dynamixel MATLAB API to preform joint position control and to extract data from the movement. By implementing `interpolate_jp()` and `measure_jp()`. With these, we are able to carry out a smooth joint transition and continuously monitor our joint positions to ensure accurate control.

B. Lab 2

For lab 2, we explored calculating the forward Kinematics of the OpenManipulator X arm. and explicitly covered the robot's joint angles to find the position and orientation of the end effector. Within lab 2 we implemented our `Dh2fk` method, this method consumes a single row of our DH table and outputs a corresponding homogeneous transformation matrix. Next with `Dh2mat` method, which processes the entire DH table. This produces a homogeneous transformation matrix that then represents from the first joint to our end effector, within this lab we then implemented symbolic FK by integrating variables into the `Dh2mat` function so we can work with unknown quantities.

C. Lab 3

Lab 3 covered Inverse Kinematics (IK) as well as Kinematic decoupling. With our process of IK we took the approach of decoupling our derivations to compensate for having more joints than our typical translational dimensions in 3D Space,

as well as simplifying the geometry needed. From there we utilized geometric identities to calculate the rest of the inverse kinematics for the manipulator (Figure 1).

IK derivation:

$$\begin{aligned}\theta_1 &= \tan^{-1} \left(\frac{P_y}{P_x} \right) \\ F4_x &= P_x - (L_4 \cos \alpha) \cos \theta_1 \\ F4_y &= P_y - (L_4 \cos \alpha) \sin \theta_1 \\ F4_z &= P_z + L_4 \sin \alpha\end{aligned}$$

Arm-Plane Calculations:

$$\begin{aligned}R_v &= F4_z - L_1 \\ R_h &= \sqrt{F4_x^2 + F4_y^2} \\ R &= \sqrt{R_h^2 + R_v^2} \\ \zeta &= \tan^{-1} \left(\frac{R_v}{R_h} \right)\end{aligned}$$

Solving Law of Cosine Angles:

$$\begin{aligned}\epsilon_{\text{int}} &= \frac{R^2 - L_2^2 - L_3^2}{-2L_2L_3} \\ \epsilon &= \tan^{-1} \left(\frac{\sqrt{1 - \epsilon_{\text{int}}^2}}{\epsilon_{\text{int}}} \right) \\ \beta_{\text{int}} &= \frac{L_3^2 - L_2^2 - R^2}{-2L_2R} \\ \beta &= \tan^{-1} \left(\frac{\sqrt{1 - \beta_{\text{int}}^2}}{\beta_{\text{int}}} \right)\end{aligned}$$

Solving Angles:

$$\begin{aligned}\text{offset} &= \tan^{-1} \left(\frac{24}{128} \right) \\ \theta_2 &= 90^\circ - \beta - \zeta - \text{offset} \\ \theta_3 &= 90^\circ - \epsilon + \text{offset} \\ \theta_4 &= \alpha - (180^\circ - \epsilon - (\beta + \zeta))\end{aligned}$$

D. Lab 4

Lab 4 went into Velocity kinematics for our serial arm, mostly with the focus of deriving and utilizing the Jacobian. This was used for singularity detection and numerical inverse kinematics. To derive the Jacobian we would later use we implemented the following. To derive the first half of the jacobian, the linear component $J(p)$, we took the partial derivative of the position vector we obtained through our Forward kinematics deprivation, with respect to each joint.

$$J_p = \begin{bmatrix} \frac{\partial p_x}{\partial \theta_1} & \frac{\partial p_x}{\partial \theta_2} & \frac{\partial p_x}{\partial \theta_3} & \frac{\partial p_x}{\partial \theta_4} \\ \frac{\partial p_y}{\partial \theta_1} & \frac{\partial p_y}{\partial \theta_2} & \frac{\partial p_y}{\partial \theta_3} & \frac{\partial p_y}{\partial \theta_4} \\ \frac{\partial p_z}{\partial \theta_1} & \frac{\partial p_z}{\partial \theta_2} & \frac{\partial p_z}{\partial \theta_3} & \frac{\partial p_z}{\partial \theta_4} \end{bmatrix}$$

For the second part of the Jacobian, the angular component or orientation $J(o)$, we

E. Lab 5 and Final Project

Lab five encompassed the final aspects of our system; Both intrinsic and extrinsic camera calibration, object detection, identification, localization, and putting everything done in previous labs together.

III. METHODOLOGY

Intrinsic Camera Calibration

We first needed to establish an accurate visual and Kinematic's foundation through our camera. This was done using MATLAB's Camera Calibration App, this helped us preform our intrinsic calibration. Multiple images of standard the checkerboard from different angles to compute a camera matrix. This included focal length, principal point, and distortion coefficients, while maintaining a mean re-projection error of less than one pixel. After verifying our intrinsic parameters, the camera is now set on the stand and lined up with our checkerboard for our is positioned within our robot's workspace for extrinsic calibration. we then acquired images and extracted the pixel coordinates of the checkerboard corners with MATLAB's `pointsToWorld()` function. This is used to compute the transformation matrix that maps the camera frame to the robot's base frame, enabling a precise conversion of the image coordinates to the real world positions.

With the Intrinsic and extrinsic camera calibration complete, an object detection and localization pipeline is implemented. A static masking is applied to focus on our desired workspace, and color thresholding techniques are used to isolate our colored balled from the background (in this case a checkerboard). Using MATLAB's `bwlabel` function, the centroids of the colored balls are determined in an image space and subsequently transformed into the robot's task space using our previously computed transformation matrix.

On the robot side, the kinematic calculations converting our joint angle data into the end-effector positions and orientation. With our forward kinematics (originally derived from our Denavit-Hartenberg parameters). Trajectory planning is employed to produced a time-parameterized way points, this ensures the arm moves smoothly and accurately between our configurations. These are executed with our calculated time intervals, and our End effector commands are interleaved to acquire our pick and places actions.

A method utilizing the pseudo-inverse of the position Jacobian was attempted to achieve a more precise and fluid motion through velocity kinematics of the system. Using the Newton-Raphson method, the difference between current and desired setpoints in task-space was caluclated at each time step, and converted to joint velocities through the inverse Jacobian and commanded. This results in a PID/gradient-descent kind of algorithm that mimics inverse kinematics by rapidly approaching the desired task space coordinates. However, since there was no clear way to enforce constraints on the final joint (commanding it at $\theta = \pi/2$), this method

was dropped in favor of standard derived inverse-kinematics equations to have more control.

IV. RESULTS

From our lab three, we hand derived the inverse kinematics for our robotic arm. We used a geometric approach, and programmed it into a MATLAB script. This script was used in our final project to determine the joint variables of the robotic arm based on the x, y, and z coordinates of the ball. These coordinates were the target location for our end effector. Below is our derivations for the inverse kinematics given the x, y, and z location of the end effectors.

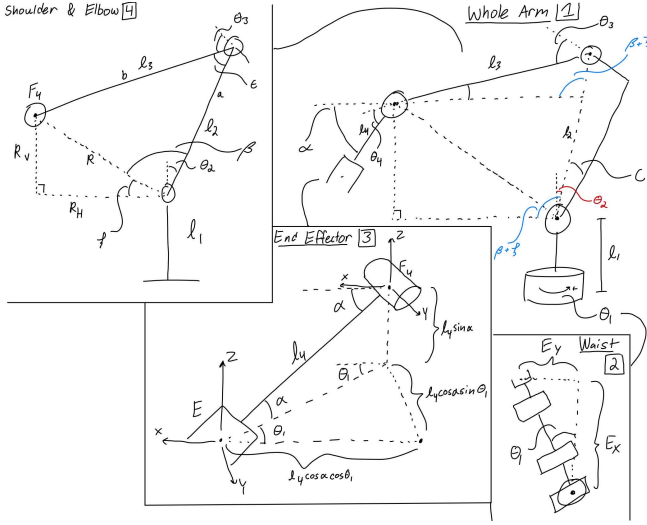


Fig. 1. Inverse Kinematic Derivation for the Robot Arm

The camera's reprojection error was found using MATLAB'S camera calibration tool. Our mean reprojection error following multiple trials of image processing and calibration was 0.37, well under our tolerance of 1.

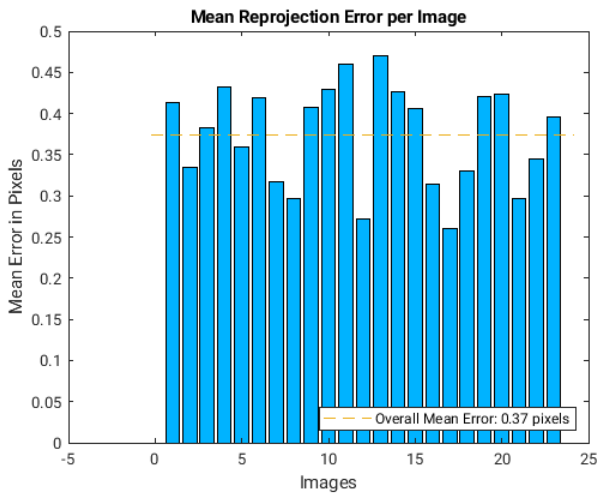


Fig. 2. Plot of Mean Reprojection Error

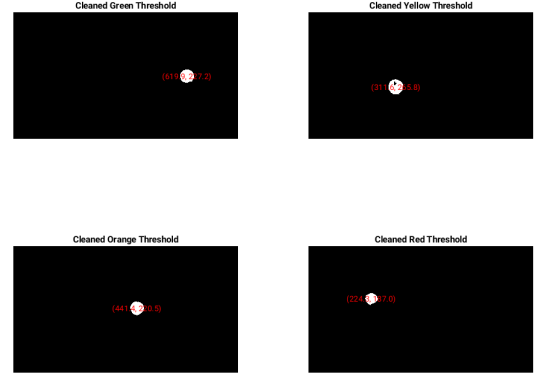


Fig. 3. Detection of Ball Centroids

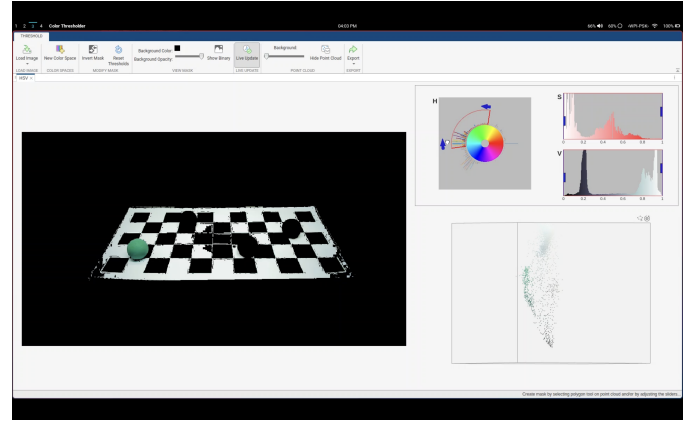


Fig. 4. Fine Tuning of HSV Thresholds in Color Thresholder App

V. DISCUSSION

A. Camera Calibration

To determine the ball's position in the robot's frame based on that of the camera's, the transformation matrices from the camera to checkerboard, and checkerboard to robot, were found and multiplied to give the robot coordinates to register.

Using over 60 images, the camera was intrinsically calibrated using MATLAB's Camera Calibration tool. Below, the figure shows the Mean Reprojection Error. This is the distance between the checkerboard points detected from the calibration image to the corresponding world point projected using the camera matrix. There was a mean error of 0.37, which was under our tolerance of 1.

B. Object Detection and Classification

One important aspect of pick-and-place is the detection and classification of objects. To tackle this challenge, a combination of MATLAB's Image Processing Toolbox and the Color Thresholder App. With the Image Processing Toolbox, a color filter is applied, resulting in a black and white binary image.

With this image, the MATLAB function **bwlabel** is used to detect the centroid of the balls in image space.

To detect color, the HSV thresholds for each color are finely tuned within the Color Threshold App, ensuring accurate detection and classification of the color of the ball. Overall, the techniques we used on the OMX not only validated our system design but also enhanced our understanding of the perception and control operate in robotic arm applications.

C. Object Localization

Localizing the objects correctly presented another task. To accurately determine the true x,y,z position of the ball's center relative to the robot coordinate system, we addressed the issue of compensating for the ball's height. Since the balls are not flat circles on the checkerboard, simply projecting upward by the ball's radius from the position obtained using **pointsToWorld()** would result in an incorrect estimate due to the perspective distortion caused by the ball's projection onto the checkerboard surface. By incorporating the radius of the balls and the height of the camera, trigonometric calculations were made to account for this adjustment, ensuring an accurate localization and coordinate for the 3D space. Without proper adjustment for the camera offset, errors may occur in the robot's end-effector positioning, leading to possible failures in picking up objects.

VI. CONCLUSION

This project demonstrated a cohesive integration of computer vision and robotics motion control for an effective pick-and-place system. The intrinsic and extrinsic camera calibration enabled accurate mapping from camera space to the OMX's workspace, while our centroid detection pipeline reliably identified and localized our objects. Our kinematic methods, including forwards and inverse kinematics, trajectory planning and Newton-Raphson refinements, provided the OMX the necessary precision and smoothness.

Throughout the process, there were many iterations of programming and testing with many challenges that appeared, especially with the consistency of real-world perception in the system like consistent and reliable centroid detection. Another difficulty was with the consistency in lighting surrounding our checkerboard. Without consistent and constant changes in lighting, the thresholds for each color often needed to be re-tuned, and without it, many errors occurred in accurate localization and detection, leading to failures with picking up the ball.

Looking ahead, there are multiple opportunities for further development. Improvements to system accuracy and efficiency can be made through more robust calibrations of the camera. This will allow for less error in object identification and localization, decreasing possibly for failures in picking up the object. Other improvements can be made to the robot's movement. A more solidified and smooth movement through the checkerboard will reduce jerkiness and speed up tasks and overall efficiency. In the future, different methods of trajectory generation and following can be investigated, such

as formulating the task as a NLP and performing trajectory optimization to account for the natural dynamics of the system while minimizing control effort, time, etc.

In conclusion, this project has provided us with a solid foundation in robotics in automation, that we can bring to our future work. The labs throughout the project and methodologies followed provide a basis for that complex tasks that these automated machines and robotics are capable of. The integration of intelligent perception and robotic manipulation presents a promising advancement in the field of robotics, offering diverse applications across industries, healthcare, and other sectors.

REFERENCES

- [1] Team 3, "RBE 3001 Final Project GitHub Repository," 2025. [Online]. Available: https://github.com/RBE3001-C25/RBE3001_C25_Team_3/releases/tag/FinalProject [Accessed: 05-Mar-2025]
- [2] Team 3, "Final Project Demonstration Video," 2025. [Online]. Available: <https://youtu.be/OrKT54cp0UI> [Accessed: 05-Mar-2025]